

ПРАКТИЧЕСКАЯ РАБОТА №2**Данные в языке Си. Базовые типы данных языка Си.**

Цель работы: изучить базовые типы данных языка Си, их размеры и диапазоны. Освоить ввод и вывод данных, управляющие последовательности и кодирование символов, а также поведение программы при переполнении и потере точности.

Общее требование: во всех заданиях подключайте `<stdio.h>`, используйте `int main(void)` и завершайте программу оператором `return 0;`. Если в задании указаны `<limits.h>`, `<stdint.h>` или `<stdbool.h>`, подключите соответствующие заголовочные файлы.

2.1. Смешанный ввод параметров.

Напишите программу, которая одним вызовом `scanf()` считывает три значения, используя разные спецификаторы: первое как десятичное, второе как шестнадцатеричное, третье как восьмеричное. Сохраните значения в три переменные и затем выведите их в десятичном виде, а также их сумму.

Таблица 2.1.1 – Тестовые данные и ожидаемое поведение

Входные данные	Выходные данные
1 2A 52	UNIT_ID: 1 UNIT_VERSION: 42 UNIT_STATUS: 42 SUM: 85

2.2. Нормализация логических флагов.

Подключите заголовочный файл `<stdbool.h>`. Напишите программу, которая считывает два целых числа и сохраняет их в две логические переменные. Выведите обе логические переменные с помощью `%d`, размер типа `bool` с помощью `sizeof()`, а также сумму двух переменных как целое число. Объясните, почему любое ненулевое входное значение превращается в 1.

Таблица 2.2.1 – Тестовые данные

Входные данные	Выходные данные
7 0	MODULE_READY: 1 FAULT_STATE: 0 BOOL_SIZE: 1 FLAGS_SUM: 1

2.3. Полный паспорт литералов.

Напишите программу, демонстрирующую все формы записи литералов языка Си. Объявите три переменные типа `int` и присвойте им литералы `10`, `010` и `0x10`, после чего выведите их значения в десятичном виде.

Выведите размеры целочисленных литералов `10`, `10u`, `10LL`, `10ULL` и вещественных литералов `0.1f`, `0.1`, `0.1L` с помощью операции `sizeof()`. Выведите результат сравнения `0.1f == 0.1` как целое число.

Объявите переменную типа `char` со значением `'A'` и выведите коды символьных литералов `'A'`, `'\x41'` и `'\101'` с помощью спецификатора `%d`, а затем выведите три размера: размер символьного литерала `'A'`, размер переменной и размер строкового литерала `"A"`.

Объясните системы счисления целочисленных литералов, роль суффиксов, почему `0.1f` не равно `0.1`, и почему символьный литерал в Си имеет размер `int`, тогда как переменная типа `char` занимает один байт.

Таблица 2.3.1 – Тестовые данные

Выходные данные
<pre>DEC_10: 10 OCT_10: 8 HEX_10: 16 INT_SUFFIX: 4 4 8 8 FLOAT_SUFFIX: 4 8 16 FLOAT_EQ: 0 CHAR_FORMS: 65 65 65 CHAR_LIT_VAR_STR: 4 1 2</pre>

2.4. Проверка согласованности диапазонов.

Подключите заголовочный файл `<limits.h>`. Выведите значения макросов `INT_MIN`, `INT_MAX` и `UINT_MAX`. Затем выведите строку `RANGE_OK`, содержащую `1`, если выражение:

```
(unsigned int)INT_MAX * 2u + 1u == UINT_MAX
```

истинно, и `0` в противном случае. В комментарии объясните, почему для этой проверки необходимо приведение к без знаковому типу.

Таблица 2.4.1 – Тестовые данные

Выходные данные
<pre>INT_MIN: -2147483648 INT_MAX: 2147483647 UINT_MAX: 4294967295 RANGE_OK: ?</pre>

2.5. Емкость фиксированных типов.

Подключите заголовочный файл `<stdint.h>`. Для каждого из типов `int8_t`, `uint8_t`, `int16_t`, `uint16_t`, `int32_t` и `uint32_t` выведите размер в байтах с помощью `sizeof()`, минимальное и максимальное значения из соответствующих макросов, а также количество различных значений, которое способен хранить тип. Количество значений вычислите как разность максимума и минимума плюс один, выполняя вычисления в типах `long long` или `unsigned long long`. Поясните, почему знаковый и без знаковые типы одинакового размера имеют одинаковое количество различных значений и чем при этом отличается распределение их диапазонов.

Таблица 2.5.1 – Тестовые данные

Выходные данные
INT8: size=1, min=-128, max=127, values=256
UINT8: size=1, min=0, max=255, values=256
INT16: size=2, min=-32768, max=32767, values=65536
UINT16: size=2, min=0, max=65535, values=65536
INT32: size=4, min=-2147483648, max=2147483647, values=4294967296
UINT32: size=4, min=0, max=4294967295, values=4294967296

2.6. Модульная арифметика регистра.

Подключите заголовочный файл `<stdint.h>`. Напишите программу, которая считывает целое число и сохраняет его в переменную типа `uint8_t`. Вычислите три результата, каждый раз присваивая значение обратно переменной типа `uint8_t`: сумму, удвоенное значение и квадрат. Выведите все три результата, приведя их к `unsigned int`. Объясните, почему результаты "сворачиваются" по модулю 256.

Таблица 2.6.1 – Тестовые данные

Входные данные	Выходные данные
250	ADD: 4 MUL2: 244 SQR: 36

2.7. Один ввод – три вещественных типа.

Напишите программу, которая считывает одно значение с помощью `scanf()` и спецификатора `%Lf` в переменную типа `long double`, после чего присваивает это значение переменным типа `double` и типа `float`. Выведите все три значения с шестью знаками после десятичной точки, используя `%.6f`, `%.6f` и `%.6Lf`. Затем выведите значения, прибавив к каждому единицу, с той же точностью.

Поясните, почему прибавление единицы "не видно" у `float`, но видно у `double` и `long double`, опираясь на примерное количество значащих цифр каждого типа.

Таблица 2.7.1 – Тестовые данные

Входные данные	Выходные данные
123456789.123457	FLOAT: 123456792.000000 DOUBLE: 123456789.123457 LDOUBLE: 123456789.123457 FLOAT+1: 123456792.000000 DOUBLE+1: 123456790.123457 LDOUBLE+1: 123456790.123457

2.8. Паспорт точности вещественных типов.

Подключите заголовочный файл `<float.h>`. Выведите размеры типов `float`, `double` и `long double` в байтах с помощью `sizeof()`, количество гарантированных значащих цифр с помощью макросов `FLT_DIG`, `DBL_DIG` и `LDBL_DIG`, а также максимальные значения с помощью макросов `FLT_MAX`, `DBL_MAX` и `LDBL_MAX`, используя спецификаторы `%e` и `%Le`. В комментарии укажите, что количество цифр показателя степени в выводе может отличаться в зависимости от системы, а на некоторых платформах `long double` может совпадать по размеру с `double`.

Таблица 2.8.1 – Тестовые данные

Выходные данные
FLOAT: size=4, digits=6, max=3.402823e+38 DOUBLE: size=8, digits=15, max=1.797693e+308 LDOUBLE: size=16, digits=18, max=1.189731e+4932

2.9. Оптическая иллюзия терминала.

Напишите программу, которая выводит слово `START`, а затем, не переходя на новую строку, превращает его на экране в слово `STOP`, используя только управляющий символ "возврат на шаг" `\b` и, при необходимости, символ пробела. Перенос строки разрешен только в самом конце вместе со звуковым сигналом `\a`. Перед запуском программы запишите ожидаемый результат и объяснение движения курсора, затем сверьте его с реальным выводом.

Таблица 2.9.1 – Тестовые данные

Выходные данные
STOP

2.10. Финальный паспорт пакета.

Подключите заголовочный файл `<stdint.h>`. Напишите программу, которая одним вызовом `scanf()` считывает идентификатор пакета в шестнадцатеричной форме `%x`, код статуса в восьмеричной форме `%o` и напряжение `%f`. Идентификатор сохраните в переменную типа `int`, код статуса приведите к типу `uint8_t`, напряжение сохраните в переменную типа `float`. Вычислите контрольную сумму в переменной `checksum` типа `uint16_t` как сумму идентификатора и кода статуса. Выведите идентификатор, числовой код статуса, соответствующий ему символ, напряжение с двумя знаками после точки и контрольную сумму.

Таблица 2.10.1 – Тестовые данные

Входные данные	Выходные данные
7 101 3.3	PACKET_ID: 7 STATUS_CODE: 65 STATUS_CHAR: A VOLTAGE: 3.30 CHECKSUM: 72